Design Note: Calendar Scheduler

React + Tailwind + Axios. Plain Date/Intl only, no calendar, date, drag-drop or state-history libraries.

1. State and history design

State lives in one **useReducer + Context** store (`store.jsx`): `{events, past, future, status}`. Events are the source of truth; recurring events are stored once as a master with a rule and a list of exceptions.

History uses the **command pattern**. A command is data, not code:

```
{ id, label, changes: [ { id, before, after } ] }
```

do applies every `after`; **undo** applies every `before`; `null` means "event does not exist", so create is `before:null` and delete is `after:null`. `exec` pushes onto `past` and clears `future`; undo/redo move the command between the two stacks. One drag commits exactly one command on pointer-up, so one drag is one undo step. Multi-event actions such as "this and following" are one command with several changes, so they also undo in one step. Ctrl+Z / Ctrl+Shift+Z and the buttons call the same functions.

2. Overlap layout

Per day, events become `{s,e}` in minutes. Sort by start (longer first on ties) and sweep once:

- Keep a **cluster** (a chain of overlapping events) and its latest end time. An event starting at or after that end closes the cluster.
- `cols[i]` stores the end of the last event in column i. An event takes the first column that is free; otherwise it opens a new one.
- On closing, every event in the cluster gets n = number of columns. Style: left = c/n, width = 1/n.

Cost is O(n log n) for the sort plus one pass. Limitation: events are not stretched into empty columns to their right, as Google Calendar does.

3. Recurrence

Only the rule is stored: `{freq, interval, until | count, nth, tz}`. `expand(event, from, to)` generates occurrences for the **visible range only**, so thousands are never stored.

- Occurrence i is computed from the master start (not from the previous one), so errors never accumulate.
- **Month ends:** monthly on the 31st uses min(day, days-in-month), so Jan 31 gives Feb 28. **Nth weekday** (every 2nd Tuesday) is derived from the weekday of the 1st; a 5th weekday that does not exist falls back to the 4th.
- **DST:** each occurrence takes the master wall-clock time (e.g. 09:00) in the rule time zone and converts it with `zoned()`, which guesses the UTC instant, reads back the wall time via Intl, and corrects twice. 09:00 stays 09:00; the UTC hour shifts.
- **Edit scopes:** *this* adds an exception and creates a standalone event; *following* truncates the old series (until = occurrence - 1 ms) and starts a new master with the remaining count; *all* shifts the master by the wall-clock delta.

4. Persistence and rollback

DummyJSON saves nothing, so the browser is the real store. Events are written to `localStorage` as `{version, events}`. On load the data is validated (schema version, required fields, end after start). If anything is wrong the key is deleted and events are re-seeded from `/todos` with a fixed rule (anchor date + id-based offsets), so seed times are identical after every refresh. Times are stored in UTC and formatted in the chosen zone with Intl.

Fake sync: a real POST to DummyJSON followed by a simulated 20% failure. The UI updates optimistically and shows Saving / Saved / Save failed. On failure the command is rolled back **only if no later command touched the same events**; otherwise it is just flagged failed. This keeps later undo steps valid (see below).

5. One hard problem: failed saves vs. undo history

Problem. Saves are optimistic, so a user can drag, edit, and drag again while an earlier save is still pending. If that first save fails, the obvious fix (undo it and delete it from history) is wrong: the later commands store `before` snapshots that include the failed change, so removing it leaves them pointing at a state that never existed, and a later undo would corrupt the event.

Fix. Each command has a unique id and its own status. On failure the reducer finds the command in `past` and checks whether any later command changed the same event ids. If none did, it is safe to apply the command `before` values and remove it. If one did, the state is left alone, the command is marked failed, and history stays consistent. Because the store always applies the latest action immediately and save results never overwrite state directly, the final state always matches the last action. The rule is covered by unit tests ("failed earlier save does not corrupt later

history").

Trade-off. In the second case the failed change stays in the local store and is only flagged, so the browser copy can be ahead of the fake server. That is acceptable here because the browser store is the source of truth.

6. Performance and accessibility

- **Drag:** pointer state in refs (no render per move); the preview is flushed once per animation frame; commit on pointer-up; Esc cancels with nothing committed. Columns and event blocks are wrapped in `memo` with stable callbacks, so during a drag only the ghost re-renders, which keeps 500+ events smooth.
- **Axios:** one instance attaches the token; on 401 all failing requests share one refresh promise, then retry once; errors are normalised to `{status, message}`.
- **Accessibility:** role="grid", arrow-key selection, M plus arrows to move, focus-trapped modals closed with Esc, and an aria-live region announcing moves.